

Single-scene U-ESFM: how it works

Method description, single-scene pipeline (from the U-ESFM project report and code)

The setting

Single-scene U-ESFM is not "training a model" in the usual sense - it is per-scene optimization, in the ESFM tradition. The network is fit from scratch to one scene's point-track matrix: rows are cameras, columns are 3D point tracks, and each filled entry is a 2D observation of some point in some camera. Nothing is learned from other scenes and nothing is transferred; the network's weights are just a convenient, equivariance-structured parameterization of that one scene's cameras and points. Ground truth is used only for evaluation, never in the loss - that is the "U" (unsupervised).

The architecture

The scene matrix is processed by a permutation-equivariant "set-of-sets" network: alternating operations that exchange information along rows (all observations in one camera), along columns (all observations of one track), and globally, so the output cannot depend on the arbitrary ordering of cameras or points. The project's contribution here is depth: where ESFM and RESfM use one block of three layers, Deep ESFM stacks many blocks and makes that trainable with three additions - residual connections through every layer so gradients survive the depth, a custom layer normalization that works on the sparse track structure, and a sparse-aware dropout for regularization. From the shared representation, one head reads off each camera's pose (and calibration-related parameters) by pooling over that camera's row, and another head reads off each 3D point by pooling over its column.

Stage 1 - initial reconstruction

The network is optimized for ten thousand epochs against the plain unsupervised reprojection loss: project every predicted 3D point through every predicted camera that observed it, and penalize the distance to the actually-measured 2D location (with a hinge mechanism handling points that project behind the camera or toward infinity). Every 250 epochs the current reconstruction is evaluated, and the best snapshot by raw reprojection error is kept. Because this loss trusts every correspondence equally, outliers - wrong feature matches, moving objects, occlusions - pull on the solution in proportion to how wrong they are. Stage 1 therefore produces a decent but outlier-contaminated reconstruction, and, crucially, a per-observation diagnostic: each correspondence's reprojection error under the best stage-1 model.

Outlier identification - statistics, not labels

The key idea is that geometric inconsistency is a usable proxy for correspondence quality. The stage-1 reprojection errors form a distribution, and outliers are flagged from that distribution alone: the MAD method thresholds at the median plus a multiple of the (robustly scaled) median absolute deviation - robust because the statistic itself barely moves when the data is heavily contaminated; the STD method thresholds at mean plus a multiple of the standard deviation - simpler, but inflatable by extreme errors; and Huber weighting makes no hard decision at all, keeping full weight below a threshold and continuously down-weighting above it. All three adapt automatically to each scene's own error scale, which is what makes them label-free.

Stage 2 - outlier-aware re-optimization

A fresh network is then optimized for another ten thousand epochs on the same scene, but now the loss only listens to what stage 1 certified: with MAD or STD the flagged correspondences are removed outright; with Huber every correspondence contributes in proportion to its weight. Freed from the outliers' pull, the reconstruction sharpens. The final best model's cameras go through robust bundle adjustment (with its own error-based filtering and re-triangulation), and the result is scored against ground truth - translation error, rotation error, registered cameras, runtime.

The fourth variant - the adaptive confidence-weighted loss

This variant replaces the two-stage process with a single joint optimization: the network also carries an outlier-classification head, and each forward pass builds its own pseudo-labels from the current reprojection-error distribution (lowest-percentile errors are confident inliers, highest-percentile confident outliers, the ambiguous middle is ignored), trains the classifier on just those with cross-entropy, and simultaneously uses the classifier's scores to down-weight the reprojection loss. It is the direct unsupervised counterpart of RESfM's supervised classifier - and it is the variant whose failure mode was diagnosed at multi-scene scale (classifier collapse to "everything is an outlier"), consistent with the project report's own reliability caveats about it.

On the name: U-ESFM rather than U-RESfM

RESfM's defining ingredient is the supervised outlier classifier; this project's primary method uses neither supervision nor (in the two-stage form) a classifier head at all. What it actually is: ESFM's unsupervised per-scene optimization, made deep, made robust by error-statistics. U-ESFM describes that accurately - and arguably "U" is even redundant, since ESFM is already unsupervised; the sharpest naming might be "Deep Robust ESFM." A reasonable scheme: U-ESFM (or DR-ESFM) for the family and the two-stage statistical method, reserving U-RESfM specifically for the adaptive-classifier variant, which genuinely is "RESfM without the labels".